

Emergent Complexity and Zero-shot Transfer via Unsupervised Environment Design (PAIRED)

백승언

10 Mar, 2024

- **Introduction**

- Importance of appropriate distribution of tasks in reinforcement learning problems

- **Protagonist Antagonist Induced Regret Environment Design(PAIRED)**

- Overview
- PAIRED

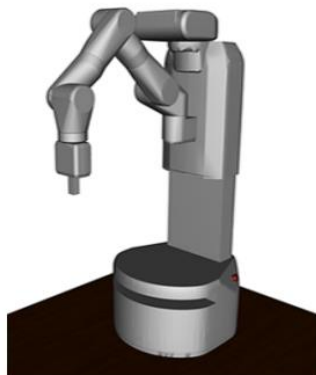
- **Experiments**

- Environments and Dataset
- Results

Introduction

Importance of appropriate distribution of tasks in RL problems

- The problem in the real world is complicated, and correctly enumerating all the edge cases relevant to an application could be impractical or impossible
 - Even if the designer of the RL method knew every edge case, specifying this distribution could take a significant amount of time and effort
- Thus, the researchers have focused on the automation of the task-generation process
 - Naïve domain randomization(DR) techniques have shown significant results in application domains such as robotics
 - Some researchers have proposed game-theoretic approaches to automate task generation process
 - The task designer player generates some tasks, and the task solver player resolves the tasks
 - The authors in this paper proposed the **Unsupervised Environment Design(UED)** paradigm



Robot-arm^[1] (DR)

[1]: <https://arxiv.org/pdf/1710.06537.pdf>



(a)



(b)



(c)

Autonomous vehicle^[2] (DR)

[2]: <https://arxiv.org/pdf/1810.02513.pdf>

Protagonist Antagonist Induced Regret Environment Design (PAIRED)

● Overview of the PAIRED

- Previous task design paradigm which is performed by the developer is error-prone
 - Furthermore, two prior approaches to automating task generation process could not generate structured tasks or adapt the difficulty of the environment to the agents' learning progress:
 - Domain randomization, minimax training
- However, a proposed model could generate feasible and challenging tasks
 - They formulated the environment generation step called UED(Unsupervised Environment Design)
 - Also, the authors designed the game of $(\pi^P, \pi^A, \vec{\theta})$ for applying minimax regret decision-making frameworks
- They proposed the objective of approximating regret and proved that the protagonist could find the optimal solution
 - $\text{REGRET}^{\vec{\theta}}(\pi^P, \pi^A) = U^{\vec{\theta}}(\pi^A) - U^{\vec{\theta}}(\pi^P)$
 - $\pi^P \in \operatorname{argmin}_{\pi^P \in \Pi^P} \left\{ \operatorname{argmax}_{\pi^A + \vec{\theta} \in \Pi \times \Theta^T} \left\{ \text{REGRET}^{\vec{\theta}}(\pi^P, \pi^A) \right\} \right\}$
- Evaluation demonstrated that PAIRED showed significant Zero-shot performance compared than other SOTA methods in complicated tasks

- **They modeled the fully specified environment and underspecified environment separately**
 - They model fully specified environments with a POMDP(Partial Observable Markov Decision Process) and they defined the utility as $U(\pi) = \sum_{t=0}^T r_t \gamma^t$, where T is a horizon
 - Also, they proposed the Underspecified POMDP(UPOMDP) as a tuple $\mathcal{M} = \langle A, O, \Theta, \mathcal{S}^{\mathcal{M}}, \mathcal{T}^{\mathcal{M}}, \mathcal{J}^{\mathcal{M}}, \mathcal{R}^{\mathcal{M}}, \gamma \rangle$
 - The only difference between a POMDP and a UMPODMP has a set Θ representing the free parameters of the environment, which can be chosen to be distinct at each time step and are incorporated into the transition function as $\mathcal{T}^{\mathcal{M}}: S \times A \times \Theta \rightarrow \Delta(S)$
- **Besides, they proposed environment generation step, which they called Unsupervised Environment Design(UED)**
 - In UED, they proposed solution to UED by specifying some environment policy, $\Lambda: \Pi \rightarrow \Delta(\Theta^T)$
 - Π is the set of possible policies and Θ^T is the set of possible sequences of environment parameters
 - Through this, they proposed PAIRED, which could be seen as an approach for approximating the environment policy, Λ , which corresponds to the decision rule minimax regret

● Protagonist Antagonist Induced Regret Environment Design(PAIRED)

- To approximate regret, they used the difference between the payoffs of two agents acting under the same env condition
 - $\text{REGRET}^{\vec{\theta}}(\pi^P, \pi^A) = U^{\vec{\theta}}(\pi^A) - U^{\vec{\theta}}(\pi^P)$, $\vec{\theta}$: the fixed params of env, π^P : protagonist agent, π^A : antagonist agent
 - $U^{\vec{\theta}}(\pi^P) = \sum_{t=1}^T r_t^{\pi^P} \gamma^t$, $U^{\vec{\theta}}(\pi^A) = \sum_{t=1}^T r_t^{\pi^A} \gamma^t$
- In PAIRED, they introduced an environment adversary that learns to control the params of the env $\vec{\theta}$, to maximize the regret of the protagonist against the antagonist
 - During training, the adversary generates the params of environment, $\vec{\theta} \sim \tilde{\Lambda}$, which both agents will play
 - The adversary and antagonist are trained to maximize the regret, while the protagonist learns to minimize regret
- They proved that the protagonist and antagonist could find optimal solution and the protagonist would be playing a minimax regret policy in this framework
 - $\pi^P \in \text{argmin}_{\pi^P \in \Pi^P} \left\{ \text{argmax}_{\pi^A + \vec{\theta} \in \Pi \times \Theta^T} \left\{ \text{REGRET}^{\vec{\theta}}(\pi^P, \pi^A) \right\} \right\}$

Algorithm 1: PAIRED.

Randomly initialize Protagonist π^P , Antagonist π^A , and Adversary $\tilde{\Lambda}$;
while not converged do
 Use adversary to generate environment parameters: $\vec{\theta} \sim \tilde{\Lambda}$. Use to create POMDP $\mathcal{M}_{\vec{\theta}}$.
 Collect Protagonist trajectory τ^P in $\mathcal{M}_{\vec{\theta}}$. Compute: $U^{\vec{\theta}}(\pi^P) = \sum_{i=0}^T r_i \gamma^i$
 Collect Antagonist trajectory τ^A in $\mathcal{M}_{\vec{\theta}}$. Compute: $U^{\vec{\theta}}(\pi^A) = \sum_{i=0}^T r_i \gamma^i$
 Compute: $\text{REGRET}^{\vec{\theta}}(\pi^P, \pi^A) = U^{\vec{\theta}}(\pi^A) - U^{\vec{\theta}}(\pi^P)$
 Train Protagonist policy π^P with RL update and reward $R(\tau^P) = -\text{REGRET}$
 Train Antagonist policy π^A with RL update and reward $R(\tau^A) = \text{REGRET}$
 Train Adversary policy $\tilde{\Lambda}$ with RL update and reward $R(\tau^{\tilde{\Lambda}}) = \text{REGRET}$
end

Pseudo code of PAIRED

Experiments

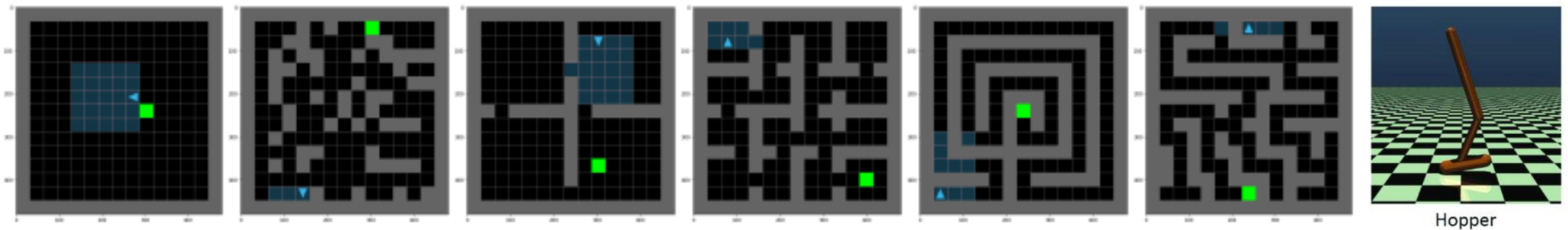
Environment and Dataset

● Key questions

- The authors performed experiments to assess whether training with PAIRED could increase the
 - (a) complexity of agents' learned behavior
 - (b) performance when transferred to novel environments

● Environment

- Partially observable navigation environment
 - The complexity of the generated environment could be represented with # of blocks, distance to the goal, and the length of the shortest path to the goal
- MuJoCo environment(Hopper task)
 - The complexity of the hopper environment could be represented with additional torques to be applied to each joint at each time step

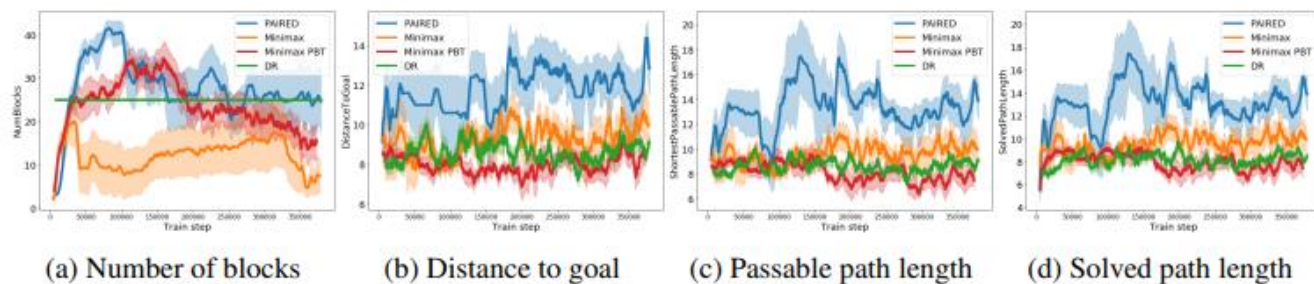


2D navigation tasks(Discrete control) and Hopper task(Continuous control)

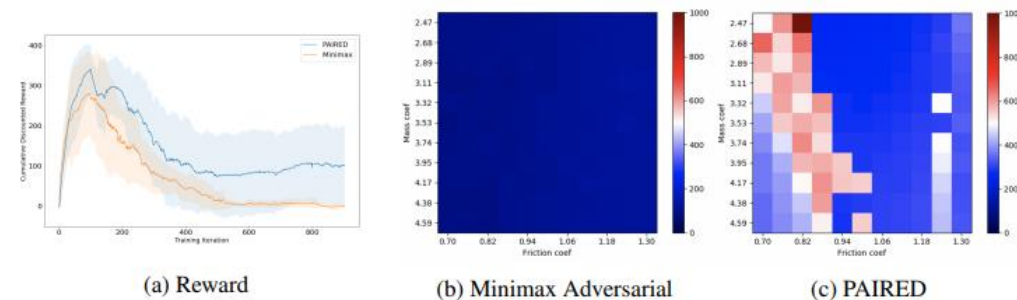
Experimental results (I)

● Comparison to previous methods

- Experiments showed significant performance compared to previous methods in learning complex behaviors
 - **DR**(Domain Randomization), **Minimax**, **Minimax PBT**(Population-Based Training)
- The authors described that the solved path could be considered a measure of the complexity of the agents' learned behavior in navigation task
 - The **DR** cannot tailor environment design to the agents' progress, so metrics remain fixed or vary randomly
 - The **minimax** training has no incentive to improve agent performance, so the length of mazes that agents can solve remains like **DR**
 - The **PAIRED** is the only method that continues to increase the passable path length to create a more challenging maze



Statistics of generated navigation environments

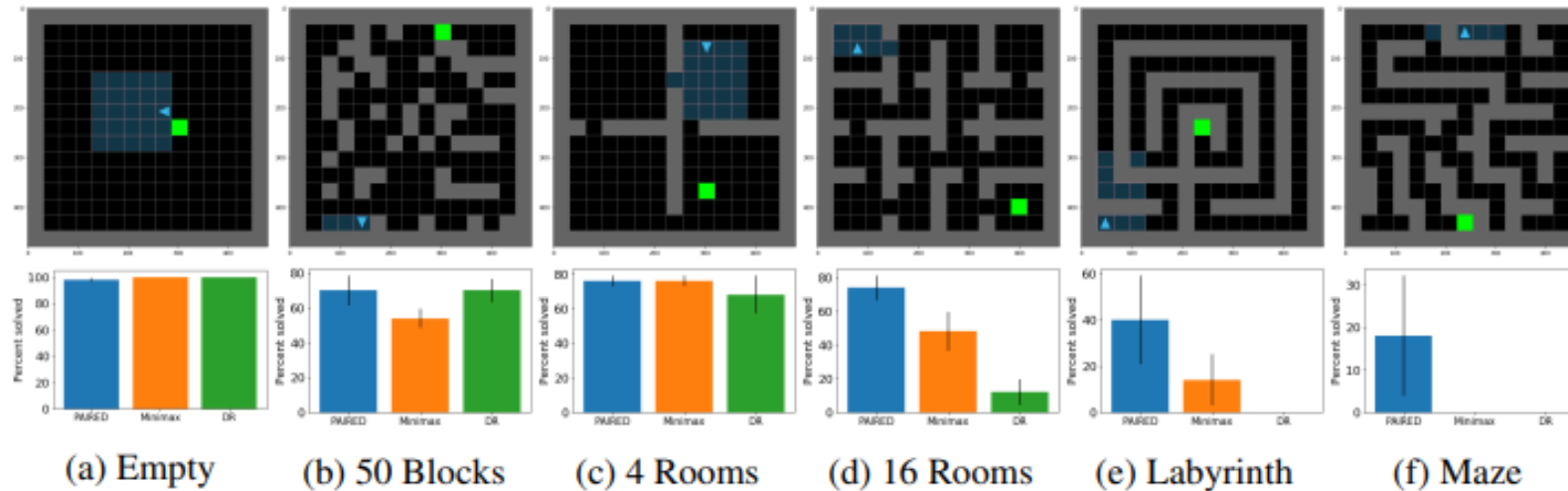


Training curves on the hopper

Experimental results (II)

● Zero-shot transfer performance

- The authors test the zero-shot transfer performance on a series of novel navigation tasks based on some criteria
 - Parameter setting
 - Number of blocks(50 Blocks)
 - Structure(4 Rooms, 16 Rooms)
 - Complex task
 - Labrinth
 - Maze
- The authors showed that as the environments increase in difficulty, so does the performance gap between the **PARIED** and the prior methods



Success ratio in navigation environments used to test zero-shot transfer

Thank you!

Q&A